information

MDPI

*Article*

# Enhancing E-Recruitment Recommendations Through Text Summarization Techniques

**Reham Hesham El-Deeb [1,*], Walid Abdelmoez [1] and Nashwa El-Bendary [2]**

[1] College of Computing and Information Technology, Arab Academy for Science, Technology & Maritime Transport, Alexandria P.O. Box 1029, Egypt; walid.abdelmoez@aast.edu
[2] College of Computing and Information Technology, Arab Academy for Science, Technology & Maritime Transport, Aswan P.O. Box 11, Egypt; nashwa.elbendary@aast.edu
* Correspondence: r.h.eldeeb@gmail.com

**Abstract:** This research aims to enhance e-recruitment systems using text summarization techniques and pretrained large language models (LLMs). A job recommender system is built with integrated text summarization. The text summarization techniques that are selected are BART, T5 (Text-to-Text Transfer Transformer), BERT, and Pegasus. Content-based recommendation is the model chosen to be implemented. The LinkedIn Job Postings dataset is used. The evaluation of the text summarization techniques is performed using ROUGE-1, ROUGE-2, and ROUGE-L. The results of this approach deduce that the recommendation does improve after text summarization. BERT outperforms other summarization techniques. Recommendation evaluations show that, for MRR, BERT performs 256.44% better, indicating relevant recommendations at the top more effectively. For RMSE, there is a 29.29% boost, indicating recommendations closer to the actual values. For MAP, a 106.46% enhancement is achieved, presenting the highest precision in recommendations. Lastly, for NDCG, there is an 83.94% increase, signifying that the most relevant recommendations are ranked higher.

**Keywords:** information retrieval; recommender systems; artificial intelligence; natural language processing; pretrained large language models; text summarization; transformers

check for updates

Academic Editor: Nirmalya Thakur

Received: 4 March 2025
Revised: 12 April 2025
Accepted: 14 April 2025
Published: 21 April 2025

**Citation:** El-Deeb, R.H.; Abdelmoez, W.; El-Bendary, N. Enhancing E-Recruitment Recommendations Through Text Summarization Techniques. *Information* **2025**, *16*, 333. https://doi.org/10.3390/info16040333

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

## 1. Introduction

Job recommender systems (JRecSyss) are tasked with connecting candidates with job opportunities to encourage real placement opportunities. When building recommender systems (RecSys) for recruitment domains, it is important to ensure that all candidates and job vacancies on the system have the opportunity to be matched as quickly as possible. Long matching times can lead to lost business opportunities, which can have financial consequences for both candidates and companies. Over time, this can erode the credibility of the recruitment services. Personalized job recommendation sites execute a variety of types of recommender systems, such as content-based [1], context-aware [2], social [3], and collaborative [4] systems and their hybrid ensembles [5]. Furthermore, most of these job recommender systems perform their suggestions based on the full profile of job seekers as well as by considering other data sources such as social networking activities, web search history, etc.

### 1.1. Recommendation Models

Recommender systems are one of the essential tools on the Internet today [6]. They are set up on many e-commerce (Amazon, eBay) and entertainment (Netflix, Spotify) platforms,

among others. They are necessary for helping users to find the most interesting products according to their interests.

To encourage the user to use recommendation systems, it is influential to recommend a variety of items to the user through the recommendation model. This expands the user's range for item selection. Also, by analyzing the user's implicit and explicit data along with those of similar groups, the recommendation system can help in the decision-making process via decreasing the selection overload by generating a list of items that matches the user's habits and characteristics.

In order for a system to present good and useful recommendations to users, it has to use efficient and accurate recommendation approaches and techniques. That is why it is vital to understand the features and abilities of different recommendation techniques.

The problem of matching jobs and candidates can be viewed in two ways: (a) finding relevant candidates for a job opening, and (b) selecting suitable jobs for a specific candidate. Regardless of the recommendation focus or approach, it is generally accepted that the term "CANDIDATE" refers to the entity that receives a set of recommended objects, and "JOB" refers to one of the objects recommended to a specific user.

A JRecSys differs from traditional recommender system domains such as e-commerce or entertainment in that it does not simply evaluate the attractiveness of the recommended products. Instead, the user–item relationship is brief and contextual and has requirements of various types. The first set of requirements relates to demographic issues, the second to technical knowledge, the third to the affinity of common interests, and the fourth to its temporal sensitivity. As a result, establishing a successful match is not a straightforward task. To design a JRecSys, the following must be defined:

- The type of recommendation approach to be used: collaborative filtering, content-based filtering, or a hybrid approach.
- The type of information to be used: job descriptions, resumes, social media data, or a combination of these.
- The domain which the recommendation will be about.
- The evaluation metrics to be used: accuracy, precision, recall, or a combination of these.
- The target audience: job seekers, employers, or both.
- The budget and resources available.

### *1.2. Transformer Pretrained Large Language Models (LLMs)*

A transformer is a neural network design that has demonstrated exceptional performance in NLP tasks. Pretrained LLMs have been trained on massive capacities of text data and have been optimized on certain downstream tasks requiring natural language processing (NLP) [7]. These pretrained transformer models have made a significant contribution to NLP research [8]. Pretrained transformer models allow researchers to optimize pretrained models for certain tasks, saving time and money in NLP research. Transfer learning is the process of applying knowledge acquired from one related task to another. By fine-tuning pretrained transformer models on novel tasks, researchers can easily implement transfer learning without the need to completely retrain the model. A comparison of Hugging Face transformer models is given below:

- Bidirectional and Auto-Regressive Transformers (BART) [abstractive]: 1.5 GB;
- Text-to-Text Transfer Transformer (T5) [abstractive]: 850 MB;
- Bidirectional Encoder Representations from Transformers (BERT) [extractive]: 1.2 GB;
- Pretraining with Extracted Gap Sentences for Abstractive Summarization (Pegasus) [abstractive]: 2.1 GB.

### 1.3. Main Contribution

The objective of this research is to provide a comprehensive overview of job recommender systems (JRecSys) and to enhance e-recruitment systems using text summarization techniques and pretrained large language models (LLMs) in order to obtain more accurate and efficient job recommendation results. The key contributions of this paper involve the following:

- Filling the research gap existent in the integration of job recommendation systems with text summarization for efficient processing of job descriptions, transformer-based models like BERT or GPT for cutting-edge semantic understanding, user behavior and preferences for personalized recommendations, and standardized evaluation metrics and scalability for handling large datasets in real time.
- Constructing a content-based job recommender system with integrated text summarization.
- Employing the LinkedIn Job Postings dataset, which was posted on Kaggle in 2023.
- Using transformers to benefit from the text summarization of pretrained large language models for BART, T5, BERT, and Pegasus architectures.
- Evaluating text summarization techniques using ROUGE-1, ROUGE-2, and ROUGE-L.
- Attaining the top-N recommendation results using the Term Frequency–Inverse Document Frequency (TF-IDF) of records and cosine similarity.
- Answering the following research question: "Can the use of transformer-based text summarization techniques improve the performance of content-based job recommender systems by reducing noise and enhancing semantic relevance in job descriptions?"

The remainder of this paper is structured as follows: First, Section 2 presents a comprehensive literature review of recommender systems for e-recruitment. In Section 3, we introduce our methodology, covering the employed dataset and proposed model. Afterwards, in Section 4, we present the obtained results and evaluation of the proposed approach starting with the experimental setup, followed by an explanation of the findings and evaluation. Finally, Section 5 concludes our research.

## 2. Related Work

*Job Recommender Systems Comparative Analysis*

A comprehensive review is performed to look into previous recommender systems, navigating their methods and approaches as well as limitations. Despite the considerable advancements that have evolved in job recommendation system research, there are plenty of opportunities for new methodologies, especially those related to text summarization, transformers, and the integration of emerging AI technologies. This comparative analysis highlights the research gaps and limitations in the existing research. The results of this analysis are shown in Table 1.

Some recent research papers include, in 2024, the study of Yingpeng Du et al. [9], who proposed an LLM-based approach for resume completion using GANs to refine low-quality resumes for better recommendations. This addressed fabricated generation and few-shot problems in LLMs. They employed three large real-world recruitment datasets and achieved improved accuracy in job recommendations by aligning low-quality resumes with high-quality ones using GANs. However, they lacked interpretability in the GAN model.

If we look at the research papers that were issued in 2023, we can see that, in [10], Atharva Patil et al. presented a detailed comprehensive survey of several filtering, machine learning, and deep learning techniques for job recommender systems. However, it lacked in-depth analysis of specific models and performance. In [11], Preetam Ghosh et al. com-

pared four approaches (content-based deterministic, LLM guided, LLM unguided, hybrid) for leveraging unstructured data in resumes and job descriptions. They highlighted explainability and contextual understanding. Also, they demonstrated superior performance in capturing nuanced criteria like cultural fit and soft skills compared to traditional methods, but they had limited scalability for large datasets.

**Table 1.** Job recommender systems analysis.

| # | Study | Method | Approach | Limitation |
|---|---|---|---|---|
| 1 | [9] | LLMs, Generative Adversarial Networks (GANs) | Job recommendation using GANs and LLMs | Lack of interpretability in the GAN model |
| 2 | [10] | Survey of AI techniques | Overview of AI-based job recommendation techniques | Lack of in-depth analysis of specific models and performance |
| 3 | [11] | LLMs, Explainable AI (XAI) | Job recommendations using explainable LLMs | Limited scalability for large datasets |
| 4 | [12] | Machine learning algorithms | General machine learning models for job recommendation | Lack of personalization and advanced feature integration |
| 5 | [13] | Semantic relational recommendation | Relation-based job recommendation using semantic matching | Poor generalization across industries or job types |
| 6 | [14] | NLP, bidirectional matching | Matching job seekers and recruiters using NLP | Limited user feedback integration and personal preferences |
| 7 | [15] | Feature fusion, representation learning | Fusion of features to improve job recommendation quality | Insufficient exploration of real-time performance |
| 8 | [16] | Multi-Criteria Decision Making (MCDM) | Ranking candidates using MCDM methods | Limited to certain industries, lack of dynamic data updates |
| 9 | [17] | Bidirectional Long Short-Term Memory (BiLSTM), convolutional neural network (CNN) | CNN and BiLSTM for HR recommendations | Lack of integration with external data sources |
| 10 | [18] | K-Means Clustering | Clustering-based recommendation system | Limited to static clustering, lacks dynamic adjustment of clusters |
| 11 | [19] | Knowledge graph | Domain-specific knowledge graph for staffing recommendations | Knowledge graph construction can be resource intensive |
| 12 | [20] | CNN, NLP | CNN-based career recommendation for Pakistani students | Limited to a single demographic (Pakistani engineering students) |
| 13 | [21] | Automation techniques | Automation in the HR recruiting process | Lack of deep learning integration |
| 14 | [22] | Deep learning, personalized attention | Personalized prediction model for job applications | Not addressing job context diversity |
| 15 | [23] | Conversational AI | Job recommendation through conversational agents | Limited to specific public sector use case |
| 16 | [24] | Content-based filtering | Job recommendations based on content similarity | Limited personalization and context understanding |
| 17 | [25] | Bidirectional Long Short-Term Memory (BiLSTM), attention mechanism | Job recommendation using LSTM and attention mechanism | Lack of interpretability and real-time feedback |
| 18 | [26] | Machine learning, data mining, RESTful API | Job recommendation through machine learning and RESTful API | Limited integration with external platforms |
| 19 | [27] | Collaborative filtering, Bayesian ranking | Collaborative filtering with Bayesian ranking | Limited scalability and not enough real-time data handling |
| 20 | [28] | Micro-service architecture | Human capital management recommendation using micro-services | Limited to specific platforms and data sources |
| 21 | [29] | Ensemble learning, gradient boosting | Ensemble learning for hybrid recommendation system | Lack of real-time adaptation and dynamic learning |

Also in 2023, S. Gadegaonkar addressed the challenge of connecting college graduates with suitable job opportunities in the IT sector. They utilized an Android application that suggests relevant jobs to users based on their skills and preferences. The recommendations are then generated through a machine learning model written in Python programming language using a content-based filtering algorithm. The application was built using Kotlin pro-

gramming language, Jetpack Compose, and Ktor, and follows Material 3 Design Principles for its user interface. The performance was measured using a dataset of job postings [12]. In [13], Denis focused on machine-learning-based recommendation systems. They used data from LinkedIn and Facebook, employing the Word-2vec method for semantic analysis and a multi-phased feature search algorithm that includes various tests. The framework utilized the Majority Voting Ensemble (MVE) concept and employed multiple machine learning methods. The results showed that the ensemble classifier performs well compared to other approaches.

Some papers published in 2022 include that of Suleiman Ali Alsaif et al. [14], who proposed a reciprocal recommendation system based on bidirectional correspondence to support both recruiters and job seekers, using machine learning to solve problems in natural language processing of text content and similarity scores. Their results showed improved prediction accuracy. They employed a resume/job offer dataset scraped from Indeed (Saudi cities). They limited integrating user feedback or personal preferences.

In [15], M. He introduced an intelligent job recommendation system. Unlike previous systems that primarily relied on textual data from job postings and resumes, this new system combined both text and structured features. It used two modules to extract hidden vectors for these features and then combined them to create a comprehensive representation of the job posting or resume. Finally, a bilinear module was employed to measure the degree of matching between the job and the resume. Experiments conducted with real-world data confirmed that this new model outperforms baseline models.

A.H. Minhas proposed an efficient ranking algorithm designed to decrease the time and cost complexities associated with candidate comparisons in Multi-Criteria Decision Making (MCDM) ranking methods. This algorithm combined matrix sorting and pruning techniques to enhance scalability. They examined the proposed algorithm applied to various datasets, including real-world recruitment, simulated DBLP, and synthetic data, which displayed decent results, highlighting its effectiveness and efficiency in real-world resume ranking procedures [16].

In [17], G. Xu addressed the challenge met by recruiters involving scanning through a large volume of resumes to find suitable candidates. He proposed a model based on recurrent convolutional neural networks. The model combined Bidirectional Long Short-Term Memory (BiLSTM) and a convolutional neural network (CNN) to obtain word-level features from both job requirements and resume text. It also included an attention mechanism to evaluate the importance of job requirements and work experiences, facilitating the effective matching of job requirements with resume content.

In 2021, B.D. Puspasari proposed an application designed to help new graduates find job vacancies that align with their criteria. It utilized the K-Means Clustering method for job recommendation. The system calculated the compatibility of the applicant's skills, salary, location, and other attributes with company requirements. The application was built using PHP, Java, jQuery, JavaScript, HTML, and CSS. The results showed that the K-Means Clustering method effectively recommended job opportunities based on applicants' personal data [18].

In [19], Y. Wang introduced a JRecSys based on skills and locations utilizing a domain-specific knowledge graph (KG). The study used real-world data from job descriptions, English CVs, and French CVs. The system was integrated within a micro-services architecture, and its performance was verified through experiments using Discounted Cumulative Gain (DCG). The system consisted of three components: Named Entity Recognition (NER) and Relation Extraction (RE) using BERT, standardization for preprocessing and semantic enrichment, and a domain-specific KG. The KG contained two types of relations computed using cosine similarity and TF-IDF.

T. Saeed focused on a Career Recommender system designed for engineering candidates. The approach employed a deep natural language processing (NLP)-based convolutional neural network (CNN) model with 512 hidden layers to enhance performance. To test the system, they used a career recommendation dataset gathered from students of different disciplines from various universities. The results showed that the proposed methodology achieved an accuracy of 84%, outperforming traditional content-based filtering techniques, which achieved 81% accuracy for career recommendations [20].

G. Rafiei introduced a novel approach to resume recommendation in recruitment systems. This approach utilized advanced machine learning (ML) algorithms to tackle the candidate ranking challenge. Notably, this work focused on the Persian language, marking a unique effort to enable HR teams to identify resumes that closely match job descriptions [21].

In [22], J. Zhu proposed a model called PANAP (Personalized-Attention Next-Application Prediction), which consisted of three modules. The first module learned job representations from textual content and metadata in an unsupervised manner. The second module learned representations for job seekers, incorporating a personalized attention mechanism that adapts to individual preferences and provides interpretability to the learned representations. The third module performed the next-application prediction task by searching for the top-K job recommendations based on similarity of representations. The model also explored the influence of geographic location on recommendations. They experimentally tested their approach on the CareerBuilder12 dataset.

V. Bellini proposed GUapp, a platform that was designed for job postings and recommendations within the Italian public administration. The recommender system in GUapp used Latent Dirichlet Allocation. To enhance the user experience, GUapp also featured a chatbot, allowing users to interact with the platform using natural language. This incremental interaction enabled users to add new requirements throughout the process [23].

T.V. Yadalam evaluated existing career recommendation systems, noting issues like cold starts, scalability, and sparsity. They explored the use of machine learning to enhance security, reliability, and transparency in the career recommendation process [24].

In 2019, A. Nigam used embedded features derived from skills to uncover latent competencies. The model was developed and deployed in a real-world job recommender system, achieving the best performance in terms of click-through rate metrics through a combination of machine learning and non-machine learning recommendations. The most successful results were obtained using Bidirectional Long Short-Term Memory (Bi-LSTM) networks with attention, which played a significant role in the job recommendations [25].

H. Jain and M. Kakkar introduced a solution to utilize machine learning and data mining techniques within a RESTful Web Server application to act as a bridge between the Android frontend and a MongoDB backend, using APIs to transfer data. Their recommender system processed data to present comprehensive job recommendations considering a wide range of factors [26].

Q. Zhou presented a Personalized Preference Collaborative Filtering (P2CF) recommendation algorithm. The P2CF clustered university graduates into groups based on their academic scores and economic conditions. It then employed the Bayesian personalized ranking (BPR) method to calculate scores for these university graduate groups in relation to available job opportunities. These results were combined with graduate preferences to recommend potential job opportunities. The P2CF had a hierarchical structure, considering both group records of job choices and individual job preferences [27].

M. Mehta presented a stateless, scalable, micro-service-based architecture for automation through recommendation. The focus was on a recommendation system for matching

candidates with jobs and vice versa. The system assigned a Profile Score to candidates and jobs, which was used as a ranking metric [28].

Y. Lin introduced an approach which encompassed ensemble learning to enhance prediction accuracy and adaptability in job recommendation algorithms. They conducted research on recommendation algorithms and the gradient promotion decision tree algorithm, building an employment recommendation algorithm. The algorithm was evaluated using a top-N list experiment [29].

## 3. Job Recommendation Systems Research Gap

Our comparative analysis identified a number of research gaps that triggered this investigation. These gaps represent interesting opportunities to improve the functionality and usability of job recommendation systems to provide users with job options that have potential and are relevant to their experience, values, and preferences. Addressing these issues could have extensive implications in the field and ultimately provide users with a better and more practical job recommendation platform. The key research gaps to address are as follows:

- Text Summarization: Despite several papers showing the recommendation of jobs based on an analysis of job descriptions, no paper claimed or even mentioned any form of output-modifying text summarization as a preprocessing step. Text summarization may need to remove noise and consider the context of a job description, to allow for recommending jobs based on job key points rather than a full job description. This could allow for more recommendations by processing the full job description faster, while allowing the job recommendation systems to consider the most salient features of a job description may produce more relevant job recommendations.
- Adoption of Transformers/Pretrained Models: Some research papers mentioned the use of deep learning models, like CNNs, BiLSTM, and GANs. However, they did not integrate modern deep learning transformer-based models like BERT or GPT, or any pretrained large language models. An appropriate transformer model typically provides results that have transformed the completion of tasks in NLP, to summarize and generate, and they are not used to their potential for recommendation systems.
- Use of User Behaviors and Preferences: Most research papers focused either on job descriptions or resumes, but a small percentage of studies focused on including user behaviors, user preferences, or historical data in their models. The embracing of user behaviors, such as past job applications, would allow for greater personalization of recommendations, leading to higher recommendation relevance. This personalization can help enhance recommendation accuracy.
- Evaluation Metrics: One of the significant findings was the lack of detailed evaluation metrics in a large number of research papers. Most studies did not clarify how the recommendations were evaluated, whether it was through accuracy, precision, recall, or other rank-aware metrics. Clear evaluation metrics are vital for comparing the effectiveness of different techniques and providing evidence for the success of the proposed model.
- Scalability of Recommendation Systems: When it comes to dealing with large datasets or a real-time recommendation environment, very few pieces of research addressed the scalability of recommendation systems and how to grow. Scalability is vital when deploying recommendation systems at a large scale.
- Explainability of Recommendations: The scarcity of studies addressing Explainable AI (XAI) is clearly noticeable. Throughout our survey, only one piece of research, on JobRecoGPT [11], embraced XAI. The level of explainability integration needs to drastically increase. Understanding why a job is recommended or why a candidate is

ranked a certain way is crucial for trust and transparency in recommendation systems. Incorporating explainability would make the systems more adequate and reliable.

*Potential Research Directions*

We have identified key research directions to improve job recommender systems. Combining text summarization approaches with feature extraction methods would generate more targeted job descriptions, which would impact the relevance of the recommendations.

Exploring the implementation of transformer-based recommender approaches such as BERT, GPT, and T5 would enrich the current job description to produce more accurate and relevant recommendations. User behavior should be considered in the recommendation process to promote personalization via analyzing user interactions with job postings in an implicit feedback approach.

Standardized evaluation protocols with offline and online metrics to validate the performance of the assessments should exist. Real-time recommendation systems should have the components necessary to easily scale data processing for the considerable extent of the job description databases. Real-time recommendation systems should contain scalability components in order to be able to analyze and process considerable data as they pertain to databases of job descriptions.

The content of the recommendation system needs to involve explainability via Explainable AI methods to instill trust in the recommendation output. A knowledge map of the research possibilities is illustrated in Figure 1.

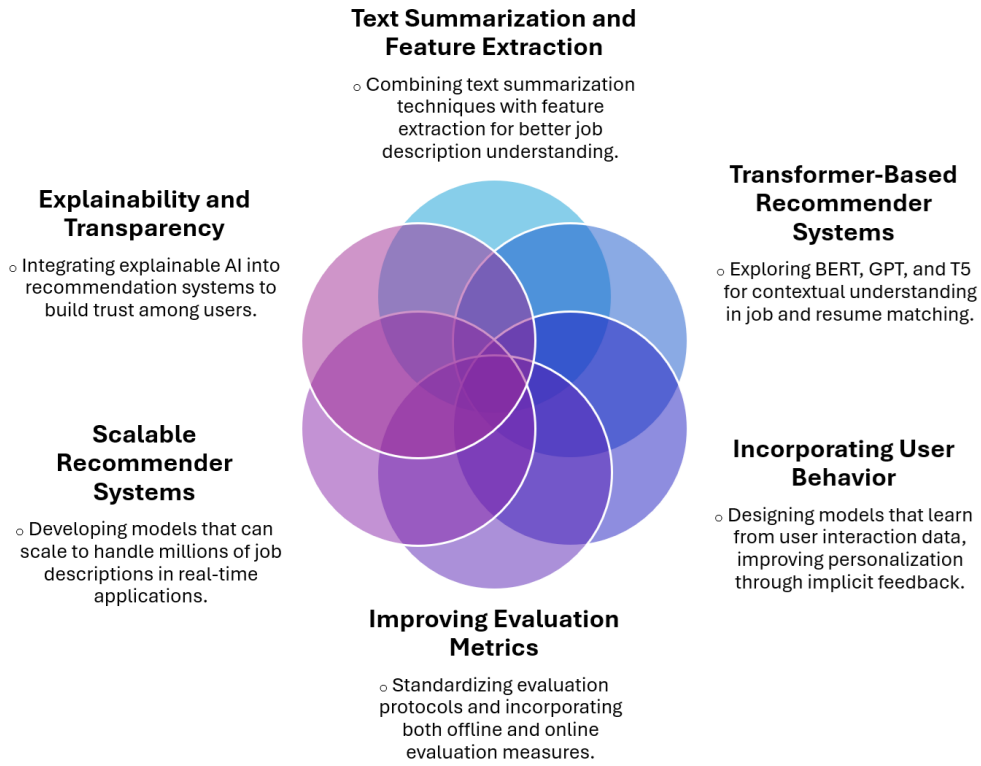


**Figure 1.** Knowledge map of potential research directions.

## 4. Methodology

### *4.1. Dataset*

As is well known, LinkedIn hosts millions of job postings, making it a valuable resource for developing and evaluating job recommendation systems. The dataset used in this research is Linkedin Job Postings, which is available on Kaggle was published in

2023 (https://www.kaggle.com/code/pratul007/decoding-the-job-market-an-in-depth-exploration/input) (accessed on 10 February 2024). This dataset contains a nearly comprehensive record of 15,000+ job postings listed over the course of 2 days. The breakdown of the files of this dataset is as follows:

- job_postings.csv;
- job_details: benefits.csv, job_industries.csv, job_skills.csv;
- company_details: companies.csv, company_industries.csv, company_specialities.csv, employee_counts.csv.

Each individual posting contains 27 valuable attributes, including the title, job description, salary, location, application URL, and work types, in addition to separate files containing the benefits, skills, and industries associated with each posting. The majority of jobs are also linked to a company, which are all listed in another CSV file containing attributes such as the company description, headquarters location, number of employees, and follower count. For this research, we used the first file "job_postings.csv". The details of this data frame are shown in Figure 2 below.

| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | job_id | 15886 non-null | int64 |
| 1 | company_id | 15520 non-null | float64 |
| 2 | title | 15886 non-null | object |
| 3 | description | 15885 non-null | object |
| 4 | max_salary | 5521 non-null | float64 |
| 5 | med_salary | 981 non-null | float64 |
| 6 | min_salary | 5521 non-null | float64 |
| 7 | pay_period | 6502 non-null | object |
| 8 | formatted_work_type | 15886 non-null | object |
| 9 | location | 15886 non-null | object |
| 10 | applies | 8700 non-null | float64 |
| 11 | original_listed_time | 15886 non-null | float64 |
| 12 | remote_allowed | 2340 non-null | float64 |
| 13 | views | 13123 non-null | float64 |
| 14 | job_posting_url | 15886 non-null | object |
| 15 | application_url | 9795 non-null | object |
| 16 | application_type | 15886 non-null | object |
| 17 | expiry | 15886 non-null | float64 |
| 18 | closed_time | 928 non-null | float64 |
| 19 | formatted_experience_level | 10984 non-null | object |
| 20 | skills_desc | 144 non-null | object |
| 21 | listed_time | 15886 non-null | float64 |
| 22 | posting_domain | 9044 non-null | object |
| 23 | sponsored | 15886 non-null | int64 |
| 24 | work_type | 15886 non-null | object |
| 25 | currency | 6502 non-null | object |
| 26 | compensation_type | 6502 non-null | object |

**Figure 2.** LinkedIn Job Postings.

The preprocessing performed on the dataset before employing it in the research included discovering the following datasets: job_postings, benefits, job_industries, job_skills, companies. Then, the job-related datasets were merged based on job_id. After that, the duplicates were removed where the same job was offered by different companies with the same description. In this scenario, we had to deal with these 'duplicated' data prior to feeding them into the model. Following this, the handling of missing data was performed, where the missing data in various columns were addressed. Categorical columns were filled with "Not Specified", numerical columns with zeros, and others based on context. Then, supplementary data were eliminated, for example, extensive company details like address and number of employees. Finally, the cleaned and processed dataset contained 10,788 entries, and a total of 6 data columns. A snapshot of the following data is shown in Figure 3 below:

- Job_title;

- Job_Description;
- BART_Description;
- T5_Description;
- BERT_Description;
- Pegasus_Description.

| title | description | Bart_desc | t5_desc | peg_desc | bert_desc |
|---|---|---|---|---|---|
| Licensed Insurance Agent | While many industries were hurt by the last few years, people still need insurance! This position is | While many industries were hurt by the last few years, people still need insurance! This position is | Sonia Sullivan-Aery has been with allstate insurance for almost eight years. she offers a very | Allstate Insurance is looking for an outstanding agent to join our team in Los Angeles, | Job Type: Full-time Pay: $19.00 - $22.00 per hour Benefits:401(k) 401(k) matchingDental |
| Sales Manager | Are you a dynamic and creative marketing professional looking to make a significant | Are you a dynamic and creative marketing professional looking to make a significant | if you're passionate about shaping brand narratives, we want to hear from you. send | Are you a passionate marketing professional looking to make a significant impact in the | If you're passionate about shaping brand narratives, developing strategic |
| Model Risk Auditor | Join Us as a Model Risk Auditor Showcase Your Financial Services Auditing | Join Us as a Model Risk Auditor Showcase Your Financial Services Auditing | as a Model Risk Auditor, you will be an integral part of our Model Risk Management (MRM) audit | Bank of America is the world's largest bank by assets, and one of the largest financial services firms in | Position Highlights:Location: New York, NY (Hybrid work arrangement)Duration: Approximately |
| Business Manager | Business ManagerFirst Baptist Church ForneyForney, Texas Reports to: Executive Pastor Job | Business ManagerFirst Baptist Church ForneyForney, Texas Reports to: Executive Pastor Job | the Business Manager maintains accurate accounting records, provides | Reporting to the Executive Pastor, the Business Manager is responsible for the day-to-day | Business ManagerFirst Baptist Church ForneyForney, Texas Reports to: Executive Pastor Job |

**Figure 3.** Cleaned and processed dataset.

For future work, and with an emphasis on scalability, we aim to implement our approach on a significantly larger dataset, such as the 1.3 M LinkedIn Jobs & Skills dataset published on Kaggle (accessed on 22 March 2025) (https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024). This dataset, which contains 1.3 million job listings scraped from LinkedIn in 2024, offers a comprehensive and diverse collection of job-related data. Leveraging this large-scale dataset will enable us to assess the effectiveness of our approach in real-world, high-volume environments, providing a more robust evaluation of its scalability and performance.

Following the recent research that experimented on LinkedIn datasets, in [30], Liala Almalki et al. proposed a job recommendation system that ensured precise and personalized recommendations by understanding the semantic relationships within job descriptions and user profiles. This research analyzed the 1.3 M LinkedIn dataset to identify trends in job market demands, skill requirements, and the most required jobs.

In [31], Paul J. Hickey et al. analyzed over 2800 LinkedIn profiles to assess career differences between genders. In [32], Sahil Panchasara et al. presented a job recommendation system that analyzed 1.3 million job listings sourced from LinkedIn in 2024. The system aimed to enhance job matching by understanding the semantic relationships within job descriptions and user profiles.

### 4.2. Proposed Model

The proposed model of this research is illustrated in Figure 4. The steps followed are described below.

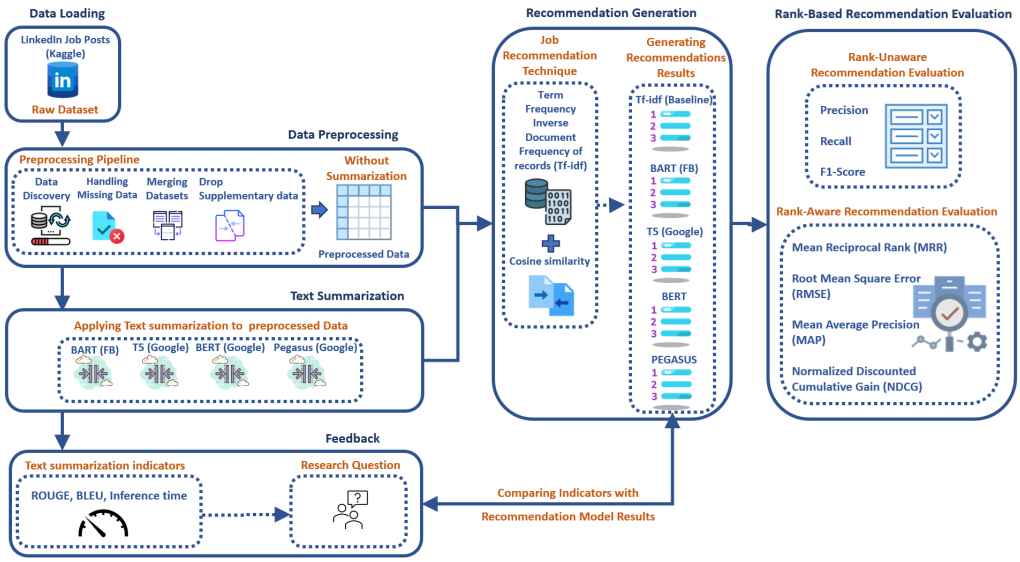


**Figure 4.** Proposed model.

- **Step 1: Data Loading:** This process begins by loading the LinkedIn Job Postings dataset obtained from Kaggle. This dataset contains numerous job details like titles, descriptions, salary, and company details.
- **Step 2: Data Preprocessing:** The data undergo a preprocessing step to understand their context and the relationships between different entities. This step involves merging appropriate tables to combine useful fields, removing duplicates if the same job is posted by different companies, and resolving any missing data by appropriately filling in null fields. This step makes the dataset clean and ready for analysis.
- **Step 3: Text Summarization:** In the "Text Summarization" module, four pretrained transformer models—BART, T5, BERT, and Pegasus—are used to generate summaries of job descriptions. Each summarization technique creates a separate column in the dataset. After summarization, the new dataset, now containing additional summarization columns, is created.
- **Step 4: Feedback:** The results of the summarization are then examined through a few standard metrics like ROUGE-1, ROUGE-2, ROUGE-L, BLEU, and inference time as an indicator of text summarization effectiveness. This brings up additional research questions: Will the transformer with the best summarization scores give the best recommendation result? Are they directly proportional? Can it be an early indicator of the upcoming recommendations?
- **Step 5: Recommendation Generation:** The aim of this step involves calculating similarities to generate a list of top-N relevant job recommendations. It starts by receiving the preprocessed data from two modules: the "Without Summarization" module and the "Text Summarization" module.
  In the "Without Summarization" module, the merged job details are directly vectorized using the TF-IDF, and similarities are calculated using cosine similarity.
  In the "Text Summarization" module, the new dataset containing the summarized job descriptions from all the pretrained models undergoes merging before vectorization. The job titles and job descriptions are firstly combined for vectorization. Then, the requested job title that needs recommendation is searched for in the dataset. Following

that, the cosine similarity is calculated between the vectorized requested job title and the complete jobs vector containing all job titles and descriptions. Finally, the top-N similar job titles are generated.

- **Step 6: Rank-Based Recommendation Evaluation:** Two types of evaluation metrics are used to evaluate the effectiveness of job recommendations. Rank-unaware metrics, such as precision, recall, and F1-score, evaluate the quality of recommendations without considering their ranking. Rank-aware metrics, such as the Mean Reciprocal Rank (MRR), Mean Average Precision (MAP), Root Mean Square Error (RMSE), and Normalized Discounted Cumulative Gain (NDCG), are used to evaluate the ranking quality of the recommendations, guaranteeing that more relevant job recommendations appear higher up the list.

### *4.3. Content-Based Recommendation System Similarity Measures*

The effectiveness [10] of recommendation systems is pivotal to user engagement and satisfaction in online platforms. With recommendation systems having a greater impact on user decision making, their assessment becomes more relevant than their technical performance alone. The five major types of metrics that together provide a comprehensive assessment framework are similarity metrics, candidate generation metrics, predictive metrics, ranking metrics, and business metrics.

Similarity metrics [33] are the cornerstone of content-based and collaborative filtering methods, offering a quantitative measure of how closely items or user preferences align. Similarity metrics offer a way to measure how similar or how far apart two items, users, or any other entity can be in a recommendation system. They are used to estimate how similar items are, which is an important component of many different recommendation methods, such as content-based filtering or collaborative filtering. The key similarity metrics used in recommendation systems are cosine similarity, Euclidean Distance, the Jaccard Index, the Hamming Distance, the Manhattan Distance, the Chebyshev Distance, Adjusted Cosine Similarity, the Pearson Correlation Coefficient, and the Spearman Rank-Order Correlation Coefficient.

### *4.4. Cosine Similarity*

CS is a measure [34] used to determine the similarity between two non-zero vectors in an n-dimensional space, capturing how closely related they are in orientation. It is calculated as the dot product of the vectors divided by the product of their magnitudes.

#### Benefits of Cosine Similarity

CS is a solid measure of contextual similarity. It compares how similar two items are based on the cosine of the angle between each of their TF-IDF vectors. This is especially useful because it can find similarities in items via cosine angle even if they have sparse data. This is very helpful for a cold-start problem, where a new user or item does not have sufficient data for either collaborative or content-based filtering recommendation approaches. CS also allows for personation recommendations because we can compare the user preference profile to the item profile, to create recommendations based on individual taste, similar to how Netflix and Amazon recommend items.

### *4.5. Term Frequency–Inverse Document Frequency (TF-IDF)*

A traditional method [35] for computing the similarity between text items functions by deriving vectors from the text. Term Frequency–Inverse Document Frequency (TF-IDF) remains one of the most commonly used IR methods for creating similarity vectors from text [36].

Benefits of TF-IDF

TF-IDF is a practical and established mechanism for determining the notionally major terms of documents, balancing term frequency (TF) scoring and inverse document frequency (IDF). It enables an emphasis on important keywords of the domain that represent items uniquely such as cuisines, menus, or movie genres. Transforming documents into unidirectional term-weighted vectors allows TF-IDF to enhance the representation of items, capturing latent signatures in the recommendation system's user modeling. It adequately distinguishes between common and uncommon words by diminishing the weight of overly common words and increasing the weight of specific terms, which supports more accurate recommendations. Furthermore, TF-IDF functions for any natural language, and is capable of handling large datasets, making it practical for global use.

### *4.6. Combining TF-IDF with Cosine Similarity*

TF-IDF and cosine similarities function together within content-based mechanisms, contributing to decoration accuracy and relevance. The combination of the established TF-IDF (content based) with the collaborative filtering mechanism (user actions) creates hybrid models which outperform either mechanism independently. The transparency offered by the use of term weighting (TF-IDF) and cosine similarity scores are more informative for user-supported models and, as such, can help users understand the reason items are recommended. TF-IDF and cosine similarity in item representation are computationally light, even with larger catalogs, when making recommendations in real time.

### *4.7. Limitations of TF-IDF with Cosine Similarity*

The main limitation of TF-IDF is that it is a purely statistical method that relies on word frequency and distribution. It treats each word as an independent feature (bag-of-words assumption), words are not influenced by neighboring words or sentence structure, and semantically related words are not linked unless they are lexically similar. When cosine similarity is combined with TF-IDF, it compares sparse vectors representing raw term weights where the similarity is based solely on term overlap, not meaning. Two semantically similar documents may have zero overlap, leading to zero cosine similarity.

Recent research that confirmed the limitations of TF-IDF and cosine similarity includes [37], where Sanchan tested BERT variants for automatic reference summary generation via centroid-based clustering, demonstrating that BERT-based models outperform traditional summarization techniques by achieving better semantic coherence despite low ROUGE scores. In [38], Shakil et al. provides a comprehensive review of abstractive summarization techniques, emphasizing how pretrained large language models like BERT address challenges such as factual consistency and cross-lingual summarization compared to older methods. In [39], Garrido-Merchan et al. empirically demonstrate that BERT consistently outperforms TF-IDF-based models in text classification tasks by capturing contextual relationships and showing independence from language-specific features. Finally, Wehnert et al., in [40], integrated Sentence-BERT embeddings with TF-IDF to achieve superior performance in legal text retrieval tasks compared to standalone TF-IDF approaches.

## 5. Results and Discussion

Throughout this research, we conducted three evaluation rounds. In the first round, we used ROUGE (Recall-Oriented Understudy for Gisting Evaluation), BLEU (Bilingual Evaluation Understudy), and inference time. ROUGE measures the overlap of unigrams, bigrams, and longer sequences (n-grams) between the generated summary and the reference summary. It focuses on recall. Higher ROUGE scores indicate better performance. BLEU is a precision-based metric. It calculates the precision of n-grams in the generated summary

relative to the reference summaries. A BLEU score value that is closer to one indicates that a summary is closer to the reference text. Although, ROUGE score is a popular evaluation metric for summarization jobs, ROUGE and BLEU can complement each other by providing a more complete evaluation of both the quality and accuracy of generated summaries. Three ROUGE variants were used in this research (ROUGE-1, ROUGE-2, and ROUGE-L), and they can be described as follows:

- ROUGE-1: Calculates the amount that the generated summary and the reference summary overlap in terms of unigrams, or single words.
- ROUGE-2: Calculates the amount that the created summary and the reference summary overlap in bigrams, or two consecutive words.
- ROUGE-L: Measures the longest common subsequence between the generated summary and the reference summary.

Inference time shows how long it takes the model to produce the summary. In general, shorter inference times are preferable, particularly for high-volume or real-time applications.

### 5.1. Text Summarization Evaluation

As shown in Table 2, the ROUGE, BLEU, and inference time scores for all four summarization techniques show that "BART Descriptions" generates the best accurate summaries since it has the highest ROUGE scores. Meanwhile, its inference time, which is the longest, might be a disadvantage in applications where timing is crucial. "T5 Descriptions" has an inference time of moderate length and moderate ROUGE scores. While it is not as accurate as "BART Descriptions", it performs better than "BERT Descriptions" and "Pegasus Descriptions". "BERT Descriptions" is less accurate than "BART Descriptions" and "T5 Descriptions", as can seen from its lower ROUGE ratings. Its inference time is, however, shorter than that of "BART Descriptions" and "T5 Descriptions". With the lowest ROUGE scores out of the four summary models, "Pegasus Descriptions" is the least accurate. However, it has the shortest inference time, which could be beneficial in scenarios where speed is more critical than accuracy.

**Table 2.** ROUGE, BLEU, and inference time scores of the different summarization techniques.

| Pretrained LLM | ROUGE-1 | ROUGE-2 | ROUGE-L | BLEU | Inference Time |
|---|---|---|---|---|---|
| BART Descriptions | 0.535832 | 0.505921 | 0.510311 | 0.108321 | 472.6042 |
| T5 Descriptions | 0.323948 | 0.287648 | 0.292804 | 0.004917 | 267.7768 |
| BERT Descriptions | 0.199262 | 0.195287 | 0.199262 | 0.000235 | 228.2956 |
| Pegasus Descriptions | 0.181876 | 0.105088 | 0.139619 | 0.000106 | 207.56 |

Since ROUGE scores are more significant in text summarization, Figure 5 illustrates these scores. "BART Descriptions" has the best summarization accuracy, as it has the highest ROUGE-1, ROUGE-2, and ROUGE-L scores. This makes it the preferred choice if accuracy is the top priority. "Pegasus Descriptions" has the shortest inference time, making it suitable for applications where response time is critical and some compromise on accuracy is acceptable. "T5 Descriptions" offers a balance between accuracy and inference time. It performs better than "BERT Descriptions" and "Pegasus Descriptions" in terms of ROUGE scores while having a relatively low inference time compared to "BART Descriptions".

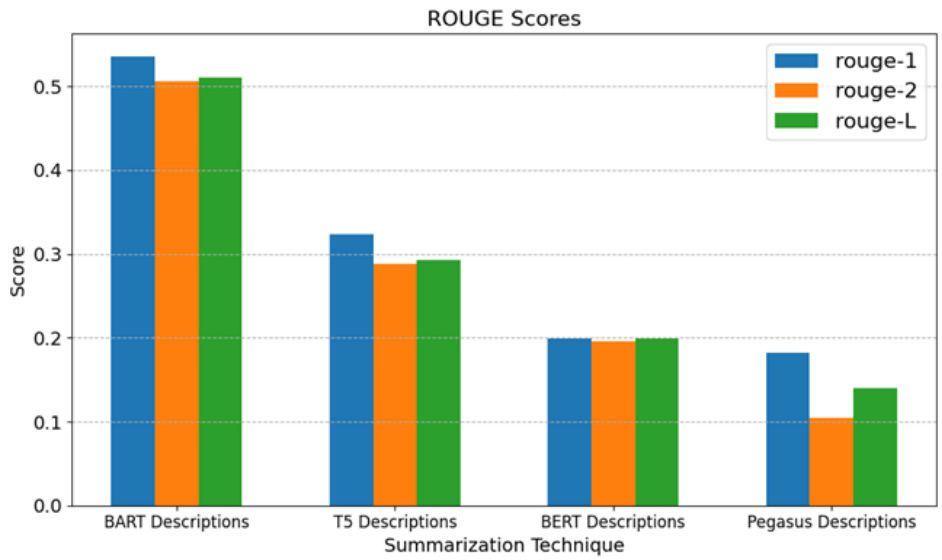


**Figure 5.** ROUGE scores for different text summarization techniques.

### *5.2. Recommendation Results Evaluation*

For the second and third evaluation rounds, recommendation evaluation measures were applied. Recommendation evaluation measures have different types, as illustrated in Figure 6 below.

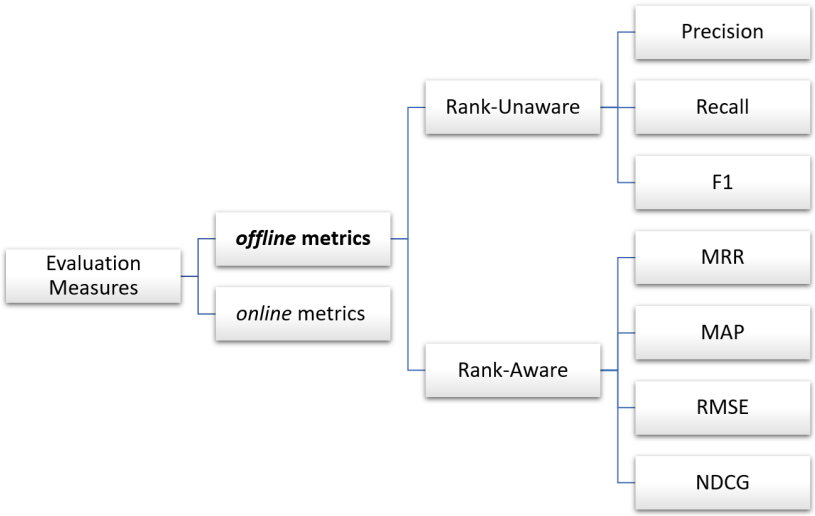


**Figure 6.** Recommendation evaluation measures.

Recommendation evaluation can be performed online via utilizing a live system and observing user-related behaviors like click-through rates and purchase conversions or offline using static data resources and matching evaluation metrics to generate numerical effectiveness measures. Offline measures examine if a specific set of pertinent results is obtained from the system when fetching objects.

These measures have two sets of measures: "rank-unaware" and "rank-aware" measures. Rank-unaware measures evaluate the quality of recommendations. These measures include precision, recall, and F1-scores, and they only reflect the number of relevant items in the top K without evaluating the ranking quality inside the list. Offline rank-aware measures help evaluate the quality of the ranking of recommendation results. It measures both the relevance of suggested items and how good the system is at placing more relevant items at the top.

### 5.3. Ranking Significance in Recommendation

Ranking is a critical function in recommendation systems, focusing on ordering items so that the most relevant ones appear at the top of a user's list. This improves user experience by making it easier and faster to find meaningful content, which, in turn, boosts the chances of user engagement. Effective ranking ensures that items closely aligned with user preferences are prioritized, directly enhancing the precision of recommendations. It also plays a vital role in evaluating system performance through rank-aware metrics, which assess how well the system brings up the most relevant content. Proper ranking not only reflects user expectations but also strengthens overall system reliability and acceptance.

### 5.4. Job Recommendation Experiment

In this setting, we assumed that Job_title is "Data Engineer". The ground truth value contains 10 values closest to this job title. Ground_truth values are as follows: ['Data Engineer', 'Senior Data Engineer', 'Software Data Engineer', 'Lead Data Engineer', 'Principal Data Engineer', 'Azure Data Engineer', 'AWS Data Engineer', 'Implementation Data Engineer', 'Data Engineer with Python AWS', 'Data Engineer/SQL Developer']. After running the recommendation algorithm, the results below were generated for each technique. To evaluate the effectiveness of each approach in recommending job titles for the job title "Data Engineer", we used a weighted ranking scheme that takes into account the following:

- **Where a job title is ranked:** The closer a recommendation is to the top of the list, the more relevant it is.
- **How many times a job title is recommended:** A job title recommended multiple times can be considered more relevant. Irrelevant titles decrease the effectiveness of the recommendation system.

Step 1: Define Ground Truth and Weighting Scheme

Ground truth: The basic order for "Data Engineer" job titles is as follows:

1. Data Engineer;
2. Senior Data Engineer;
3. Software Data Engineer;
4. Lead Data Engineer;
5. Principal Data Engineer;
6. Azure Data Engineer;
7. AWS Data Engineer;
8. Implementation Data Engineer;
9. Data Engineer with Python AWS;
10. Data Engineer/SQL Developer.

We assigned higher weights to jobs that appear at higher ranks and assigned smaller weights to jobs that appear at lower ranks. The Inverse Rank Weighting method (1/Rank) was used.

Step 2: Assign Weights to Ground Truth Jobs

Weight for Rank 1 (Data Engineer): 1/1 = 1.0;
Weight for Rank 2 (Senior Data Engineer): 1/2 = 0.5;
Weight for Rank 3 (Software Data Engineer): 1/3 = 0.33;
Weight for Rank 4 (Lead Data Engineer): 1/4 = 0.25;
Weight for Rank 5 (Principal Data Engineer): 1/5 = 0.20;
Weight for Rank 6 (Azure Data Engineer): 1/6 = 0.17;
Weight for Rank 7 (AWS Data Engineer): 1/7 = 0.14;

Weight for Rank 8 (Implementation Data Engineer): 1/8 = 0.125;
Weight for Rank 9 (Data Engineer with Python AWS): 1/9 = 0.11;
Weight for Rank 10 (Data Engineer/SQL Developer): 1/10 = 0.10.

Step 3: Compare Each Approach to the Ground Truth

Then, we compared each approach's ranking for the "Data Engineer" job title and calculated the effectiveness based on the above criteria, which are shown in Tables 3, 4, 5, 6 and 7, respectively.

*Approach 1: TF-IDF*

**Table 3.** TF-IDF recommendations scoring.

| Rank | Job Title | Ground Truth Rank | Weight Assigned | Relevance |
|---|---|---|---|---|
| 1 | Analytics Data Solutions Architect | Irrelevant | 0 | Irrelevant |
| 2 | Lead Data Engineer | 4 | 0.25 | *Relevant* |
| 3 | Senior Data Architect | Irrelevant | 0 | Irrelevant |
| 4 | IT Data Warehouse Analyst | Irrelevant | 0 | Irrelevant |
| 5 | Data Engineer | 1 | 1.0 | *Relevant* |
| 6 | Senior Manager | Irrelevant | 0 | Irrelevant |
| 7 | Senior Data Engineer | 2 | 0.5 | *Relevant* |
| 8 | Data Engineer | 1 | 1.0 | Relevant |
| 9 | Enterprise Data Management Admin | Irrelevant | 0 | Irrelevant |
| 10 | Lead Data Engineer | 4 | 0.25 | *Relevant* |

The total score for TF-IDF is 1.0 + 0.25 + 0.5 + 1.0 + 0.25 = 3.0.
The final recommended jobs are Analytics Data Solutions Architect, Lead Data Engineer, Senior Data Architect, IT Data Warehouse Analyst, Data Engineer, Senior Manager, Senior Data Engineer, and Enterprise Data Management Admin.

*Approach 2: BART*

**Table 4.** BART recommendations scoring.

| Rank | Job Title | Ground Truth Rank | Weight Assigned | Relevance |
|---|---|---|---|---|
| 1 | Data Engineer IV | Irrelevant | 0 | Irrelevant |
| 2 | Senior Manager | Irrelevant | 0 | Irrelevant |
| 3 | Data Engineer | 1 | 1.0 | *Relevant* |
| 4 | Data Engineer | 1 | 1.0 | *Relevant* |
| 5 | Data Engineering Product Lead | Irrelevant | 0 | Irrelevant |
| 6 | Lead Data Engineer | 4 | 0.25 | *Relevant* |
| 7 | Database Engineer | Irrelevant | 0 | Irrelevant |
| 8 | Senior Data Engineer | 2 | 0.5 | *Relevant* |
| 9 | Database Management Analyst | Irrelevant | 0 | Irrelevant |
| 10 | Data Engineer | 1 | 1.0 | *Relevant* |

The total score for BART is 1.0 + 1.0 + 0.25 + 0.5 + 1.0 = 3.75.
The final recommended jobs are Data Engineer IV, Senior Manager, Data Engineer, Data Engineering Product Lead, Lead Data Engineer, Database Engineer, Senior Data Engineer, and Database Management Analyst.

*Approach 3: T5*

**Table 5.** T5 recommendations scoring.

| Rank | Job Title | Ground Truth Rank | Weight Assigned | Relevance |
|---|---|---|---|---|
| 1 | Lead Data Architect | Irrelevant | 0 | Irrelevant |
| 2 | Senior Manager | Irrelevant | 0 | Irrelevant |
| 3 | Data Engineer | 1 | 1.0 | *Relevant* |
| 4 | Data Governance Specialist | Irrelevant | 0 | Irrelevant |
| 5 | Database Engineer | Irrelevant | 0 | Irrelevant |
| 6 | Data Analyst | Irrelevant | 0 | Irrelevant |
| 7 | Enterprise Data Architect | Irrelevant | 0 | Irrelevant |
| 8 | Data Analytics Solutions Engineer | Irrelevant | 0 | Irrelevant |
| 9 | Global Data Insights Analyst | Irrelevant | 0 | Irrelevant |
| 10 | Senior Data Engineer | 2 | 0.5 | *Relevant* |

The total score for T5 is 1.0 + 0.5 = 1.5.

The final recommended jobs are Lead Data Architect, Senior Manager, Data Engineer, Data Governance Specialist, Database Engineer, Data Analyst, Enterprise Data Architect, Data Analytics Solutions Engineer, Global Data Insights Analyst, and Senior Data Engineer.

*Approach 4: BERT*

**Table 6.** BERT recommendations scoring.

| Rank | Job Title | Ground Truth Rank | Weight Assigned | Relevance |
|---|---|---|---|---|
| 1 | Data Engineer | 1 | 1.0 | *Relevant* |
| 2 | Data Engineer | 1 | 1.0 | *Relevant* |
| 3 | Data Engineer | 1 | 1.0 | *Relevant* |
| 4 | Lead Data Engineer | 4 | 0.25 | *Relevant* |
| 5 | Lead Data Engineer | 4 | 0.25 | *Relevant* |
| 6 | Scala Developer | Irrelevant | 0 | Irrelevant |
| 7 | Data Engineer | 1 | 1.0 | *Relevant* |
| 8 | Data Engineer | 1 | 1.0 | *Relevant* |
| 9 | Senior Data Engineer | 2 | 0.5 | *Relevant* |
| 10 | Senior Information Technology Program Manager | Irrelevant | 0 | Irrelevant |

The total score for BERT is 1.0 + 1.0 + 1.0 + 0.25 + 0.25 + 1.0 + 1.0 + 0.5 = 6.0.

The final recommended jobs are Data Engineer, Lead Data Engineer, Scala Developer, Senior Data Engineer, and Senior Information Technology Program Manager.

*Approach 5: Pegasus*

**Table 7.** Pegasus recommendations scoring.

| Rank | Job Title | Ground Truth Rank | Weight Assigned | Relevance |
|---|---|---|---|---|
| 1 | Data Engineer | 1 | 1.0 | *Relevant* |
| 2 | Data Analytics Engineer | Irrelevant | 0 | Irrelevant |
| 3 | Data Engineer | 1 | 1.0 | *Relevant* |
| 4 | Senior Data Engineer | 2 | 0.5 | *Relevant* |
| 5 | Sr. Data Engineer | 2 | 0.5 | *Relevant* |
| 6 | GCP Data lead/Architect | Irrelevant | 0 | Irrelevant |
| 7 | Lead Data Engineer | 4 | 0.25 | *Relevant* |
| 8 | Data Engineer | 1 | 1.0 | *Relevant* |
| 9 | Information Technology Infrastructure Engineer | Irrelevant | 0 | Irrelevant |
| 10 | Staff Data Engineer and Team Lead | Irrelevant | 0 | Irrelevant |

The total score for Pegasus is 1.0 + 1.0 + 0.5 + 0.5 + 1.0 = 4.0.

The final recommended jobs are Data Engineer, Data Analytics Engineer, Senior Data Engineer, Sr. Data Engineer, GCP Data lead/Architect, Lead Data Engineer, Information Technology Infrastructure Engineer, and Staff Data Engineer and Team Lead.

We can clearly see the significance of BERT recommendation results in comparison to the rest of the techniques. It produced the highest score (6.0), indicating it provided the most relevant recommendations in the correct order, with important job titles ranked higher and relevant job titles presented multiple times. We can also observe the inefficiency of the T5 results.

#### 5.4.1. Rank-Unaware Evaluation

Precision measures the proportion of correct job title recommendations out of all recommendations made, while recall measures the correct recommendations relative to the total number of actual (ground truth) job titles.

In this experiment, BERT significantly outperformed other techniques in both metrics, whereas T5 showed weaker performance. Additionally, using rank-unaware evaluation methods proved ineffective, as they did not account for the order or frequency of correct recommendations. This approach offered limited insight, as the results across techniques appeared similar, failing to accurately reflect the quality or effectiveness of each recommendation method.

#### 5.4.2. Rank-Aware Evaluation

However, when executing rank-aware evaluation, as shown in Table 8, a more accurate evaluation was depicted. Considering TF-IDF as the baseline for comparing the summarization techniques, we deduced the following:

**Table 8.** Rank-aware evaluation results.

| Technique | Mean Reciprocal Rank | Root Mean Square Error | Mean Average Precision | Normalized Discounted Cumulative Gain |
|---|---|---|---|---|
| TF-IDF | 0.1543 | 5.099 | 0.4657 | 0.4304 |
| BART | 0.1292 | 5.1769 | 0.4667 | 0.4163 |
| T5 | 0.0433 | 5.831 | 0.2667 | 0.1737 |
| BERT | *0.55* | *3.6056* | *0.9617* | *0.7917* |
| Pegasus | 0.106 | 4.062 | 0.4405 | 0.3476 |

For Mean Reciprocal Rank (MRR), BERT was the best technique, giving 0.5500, and T5 was the worst, giving 0.0433. BERT outperformed the others, indicating that it brings relevant recommendations to the top more effectively, as illustrated in Figure 7.

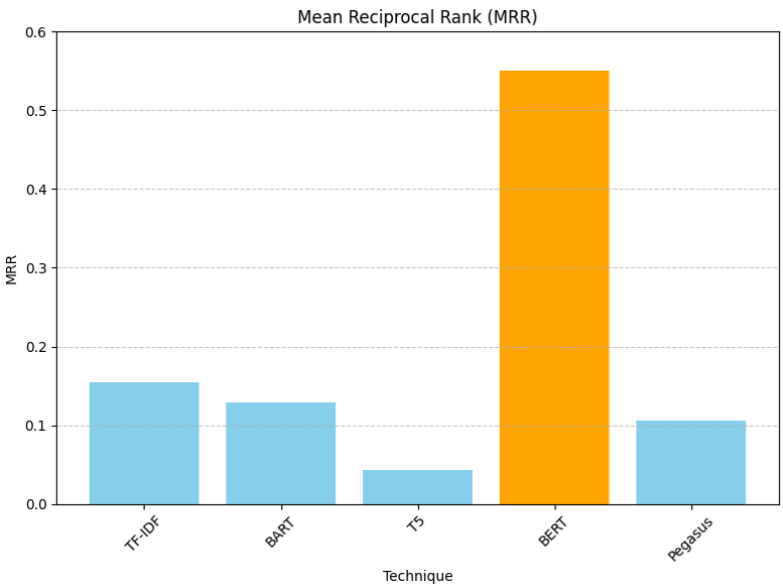


**Figure 7.** Rank-aware Mean Reciprocal Rank evaluation results.

For the Root Mean Square Error (RMSE), BERT gave the lowest RMSE (3.6056), and T5 gave 5.8310, which was the worst score, as shown below in Figure 8. BERT's score indicated that its recommendations are closer to the actual values.

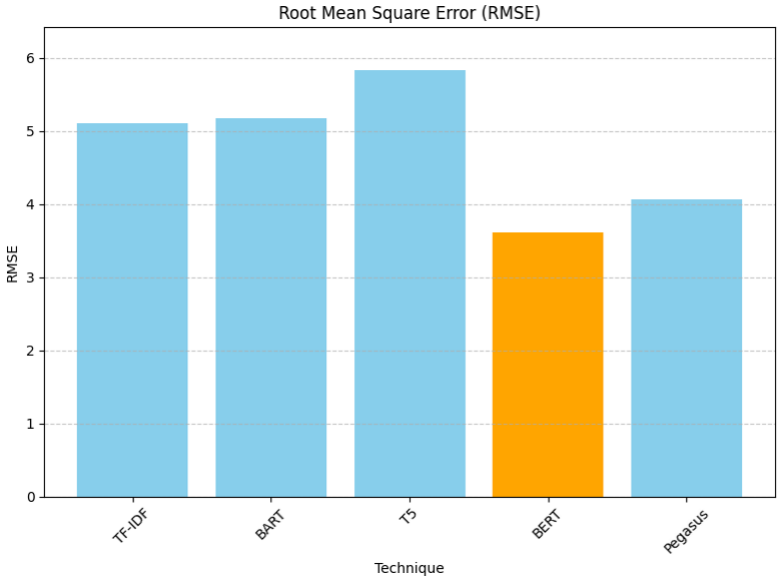


**Figure 8.** Rank-aware Root Mean Square Error evaluation results.

As depicted in Figure 9, the Mean Average Precision (MAP) scores indicated that BERT presented the highest value and T5 the lowest. BERT again showed the highest precision in recommendations.

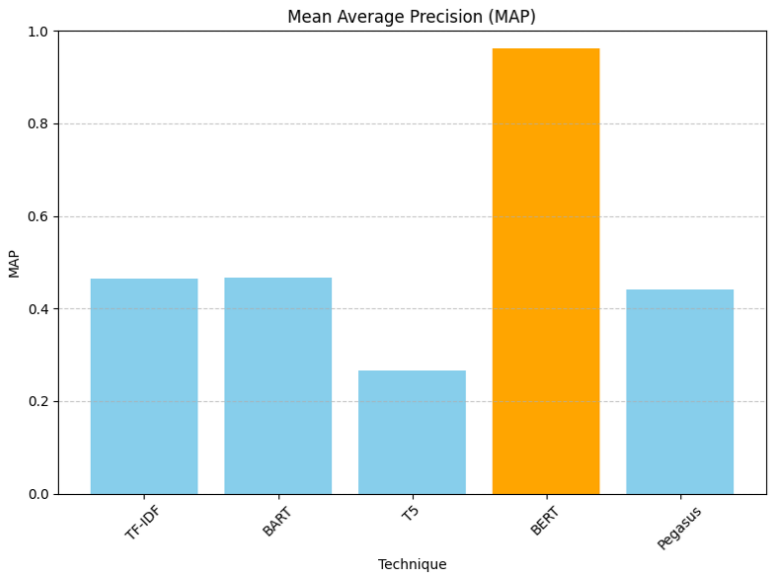


**Figure 9.** Rank-aware Mean Average Precision evaluation results.

Lastly, in terms of the Normalized Discounted Cumulative Gain (NDCG) results, illustrated in Figure 10 below, BERT presented 0.7917, while the lowest was presented by T5 with 0.1737. BERT's score indicated that the most relevant recommendations are ranked higher.

Therefore, if we look at the overall performance, we can infer that BERT's superior performance can be attributed to its architecture and pretraining on a large corpus, which makes it effective at understanding and generating human-like text. BART and Pegasus performed moderately, better than T5 but not as well as BERT. BART was closer to the baseline

TF-IDF in terms of performance. T5's lower performance may indicate that its summaries might be losing essential context needed for effective TF-IDF-based recommendations.

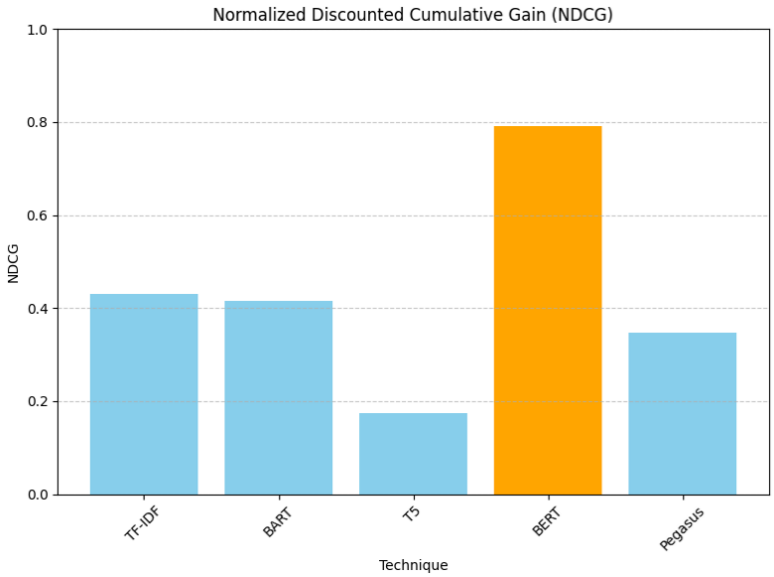


**Figure 10.** Rank-aware Normalized Discounted Cumulative Gain evaluation results.

Also, the ROUGE scores that evaluated the text summarization accuracy were not a correct indicator. They displayed that "BART Descriptions" has the best summarization accuracy, as it has the highest ROUGE-1, ROUGE-2, and ROUGE-L scores. "Pegasus Descriptions" has the shortest inference time. However, in reality, for this application, accuracy and inference time are not the features we should be evaluating with. BERT achieved average ROUGE scores; however, it outperformed all others in all recommendation evaluation results.

### *5.5. Why Did BERT Perform Better Despite Its ROUGE Scores?*

ROUGE is a surface-level metric that favors lexical similarity over semantic understanding. BERT-based summarization models use transformer-based embeddings and attention to generate semantically meaningful summaries, which preserve context and intent. So, when a BERT-based summarizer produces a semantically accurate but lexically different summary, ROUGE scores may be low even if the summary is meaningful and concise. So why does BERT-based summarization work with TF-IDF and cosine similarity in content-based recommender systems? Simply, it is because job descriptions are often verbose. Summarization compresses the description to its essential keywords. This compression removes irrelevant terms that can skew TF-IDF weights and cosine similarity. A BERT-generated summary is more semantically compact, so TF-IDF vectors represent more coherent topical vectors, leading to better cosine similarity results.

## 6. Limitations

The research has some limitations. First, the dataset is made up of about 15,000 job postings and was obtained from LinkedIn. Thus, the jobs contained in the data may not be representative of many job markets, industries, and regions, which may reduce the generalizability of the results. Second, cosine similarity was used to assess the textual relevance in job descriptions. While cosine similarity is appropriate for assessing exact matches of text, it has no awareness of semantics and thus does not represent subtle contextual relationships between job descriptions. While some of this challenge can be resolved using more recent semantic models like BERT embeddings, which allow for

assessment of contextual similarities, these models were not specifically examined in this study.

## 7. Conclusions and Future Work

BERT is the best-performing summarization technique for enhancing TF-IDF-based job description recommendations, significantly improving recommendation accuracy and relevance over other techniques and the baseline. BERT outperforms the baseline across all metrics, indicating that the summarization process enhances the quality of the job description, leading to better recommendations.

Based on the results, it is reasonable to conclude that the recommendation indeed becomes better after text summarization. The summarization process helps distill the essential information from the job descriptions, enabling the recommendation system to make more accurate and relevant recommendations. However, the degree of improvement may vary depending on the summarization technique used, with BERT showing the most significant enhancement in recommendation quality.

Recommendation evaluations showed that, for MRR, BERT performed 256.44% better, indicating relevant recommendations at the top more effectively. For RMSE, there was a 29.29% boost, indicating recommendations closer to the actual values. For MAP, there was a 106.46% enhancement, presenting the highest precision in recommendations. Lastly, for NDCG, there was an 83.94% increase, signifying that the most relevant recommendations are ranked higher.

Further research will explore the integration of more language models, such as GPT-4, to see whether these can further improve text summarization and recommendation accuracy. Furthermore, an exploration of a larger LinkedIn dataset specifically testing the scalability of the proposed recommendation mechanism against the upcoming 1.3 M LinkedIn Job Posts dataset (2024) will be performed. Our main intention with the consequent work is not to breed the same kind of job postings that were exhibited in this study, but rather test the effectiveness of various large language models (LLMs) such as BERT embeddings to improve the job recommendation process. We believe that we can enhance job recommendation accuracy when using LLMs, as a result of their ability to contextualize words within the experience of practitioners' decisions. At this moment, we also intend to complement the overall recommendation experience and quality by introducing explainability modules such as SHAP and LIME. It is our goal to both improve recommendation quality and introduce a mechanism for charting the use and metrics of job recommendation decisions. Ultimately, we want to optimize the job recommendation process with accuracy, validity, and trustworthiness in the way it concludes a recommendation for any job posting.

**Author Contributions:** Conceptualization, W.A., N.E.-B. and R.H.E.-D. methodology, N.E.-B. and R.H.E.-D.; software, R.H.E.-D.; validation, W.A. and N.E.-B.; formal analysis, N.E.-B. and R.H.E.-D.; investigation, W.A. and R.H.E.-D.; resources, W.A., N.E.-B. and R.H.E.-D.; data curation, R.H.E.-D.; writing—original draft preparation, R.H.E.-D.; writing—review and editing, W.A. and N.E.-B.; visualization, R.H.E.-D.; supervision, W.A. and N.E.-B.; project administration, W.A. and N.E.-B.; All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** Dataset available at: https://www.kaggle.com/code/pratul007/decoding-the-job-market-an-in-depth-exploration/input (accessed on 4 March 2025).

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References


1. Deldjoo, Y.; Schedl, M.; Cremonesi, P.; Pasi, G. Recommender Systems Leveraging Multimedia Content. *Acm Comput. Surv.* **2020**, *53*, 1–38. [CrossRef]
2. Kulkarni, S.; Rodd, S.F. Context Aware Recommendation Systems: A review of the state of the art techniques. *Comput. Sci. Rev.* **2020**, *37*, 100255. [CrossRef]
3. Shokeen, J.; Rana, C. A study on features of social recommender systems. *Artif. Intell. Rev.* **2019**, *53*, 965–988. [CrossRef]
4. Alhijawi, B.; Kilani, Y. A collaborative filtering recommender system using genetic algorithm. *Inf. Process. Manag.* **2020**, *57*, 102310. [CrossRef]
5. Çano, E.; Morisio, M. Hybrid recommender systems: A systematic literature review. *Intell. Data Anal.* **2017**, *21*, 1487–1524. [CrossRef]
6. Aggarwal, C.C. *Recommender Systems*; Springer: Cham, Switzerland, 2016; Volume 1.
7. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention Is All You Need. *arXiv* **2023**, arXiv:1706.03762.
8. Lalramhluna, R.; Dash, S.; Pakray, D. MizBERT: A Mizo BERT Model. *Acm Trans. Asian-Low-Resour. Lang. Inf. Process.* **2024**, *23*, 1–14. [CrossRef]
9. Du, Y.; Luo, D.; Yan, R.; Wang, X.; Liu, H.; Zhu, H.; Song, Y.; Zhang, J. Enhancing Job Recommendation through LLM-Based Generative Adversarial Networks. *Proc. AAAI Conf. Artif. Intell.* **2024**, *38*, 8363–8371. [CrossRef]
10. Patil, A.; Suwalka, D.; Kumar, A.; Rai, G.; Saha, J. A Survey on Artificial Intelligence (AI) based Job Recommendation Systems. In Proceedings of the 2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), Erode, India, 23–25 March 2023; IEEE: New York, NY, USA, 2023; pp. 730–737. [CrossRef]
11. Ghosh, P.; Sadaphal, V. JobRecoGPT—Explainable job recommendations using LLMs. *arXiv* **2023**, arXiv:2309.11805. [CrossRef]
12. Gadegaonkar, S.; Lakhwani, D.; Marwaha, S.; Salunke, P.A. Job Recommendation System using Machine Learning. In Proceedings of the 2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS), Coimbatore, India, 2–4 February 2023; IEEE: New York, NY, USA, 2023; pp. 596–603. [CrossRef]
13. Denis, R.; Peter Jose, P.; Sushma Margaret, A. Performance Analysis of Machine Learning—Semantic Relational Approach based Job Recommendation System. In Proceedings of the 2023 10th International Conference on Computing for Sustainable Global Development (INDIACom), New Delhi, India, 15–17 March 2023; pp. 1478–1486.
14. Alsaif, S.A.; Sassi Hidri, M.; Ferjani, I.; Eleraky, H.A.; Hidri, A. NLP-Based Bi-Directional Recommendation System: Towards Recommending Jobs to Job Seekers and Resumes to Recruiters. *Big Data Cogn. Comput.* **2022**, *6*, 147. [CrossRef]
15. He, M.; Zhu, Y.; Lv, N.; He, R. A Feature Fusion-based Representation Learning Model for Job Recommendation. In Proceedings of the 2022 2nd International Conference on Consumer Electronics and Computer Engineering (ICCECE), Guangzhou, China, 14–16 January 2022; IEEE: New York, NY, USA, 2022; pp. 791–794. [CrossRef]
16. Minhas, A.H.; Shaiq, M.D.; Qureshi, S.A.; Cheema, M.D.A.; Hussain, S.; Khan, K.U. An Efficient Algorithm for Ranking Candidates in E-Recruitment System. In Proceedings of the 2022 16th International Conference on Ubiquitous Information Management and Communication (IMCOM), Seoul, Republic of Korea, 3–5 January 2022; IEEE: New York, NY, USA, 2022; pp. 1–8. [CrossRef]
17. Xu, G. Human Resource Recommendation Based on Recurrent Convolutional Neural Network. In Proceedings of the 2022 2nd International Conference on Consumer Electronics and Computer Engineering (ICCECE), Guangzhou, China, 14–16 January 2022; IEEE: New York, NY, USA, 2022; pp. 54–58. [CrossRef]
18. Puspasari, B.D.; Damayanti, L.L.; Pramono, A.; Darmawan, A.K. Implementation K-Means Clustering Method in Job Recommendation System. In Proceedings of the 2021 7th International Conference on Electrical, Electronics and Information Engineering (ICEEIE), Malang, Indonesia, 2 October 2021; IEEE: New York, NY, USA, 2021; pp. 1–6. [CrossRef]
19. Wang, Y.; Allouache, Y.; Joubert, C. A Staffing Recommender System based on Domain-Specific Knowledge Graph. In Proceedings of the 2021 Eighth International Conference on Social Network Analysis, Management and Security (SNAMS), Gandia, Spain, 6–9 December 2021; IEEE: New York, NY, USA, 2021; pp. 1–6. [CrossRef]
20. Saeed, T.; Sufian, M.; Ali, M.; Rehman, A.U. Convolutional Neural Network Based Career Recommender System for Pakistani Engineering Students. In Proceedings of the 2021 International Conference on Innovative Computing (ICIC), Lahore, Pakistan, 9–10 November 2021; IEEE: New York, NY, USA, 2021; pp. 1–10.[CrossRef]
21. Rafiei, G.; Farahani, B.; Kamandi, A. Towards Automating the Human Resource Recruiting Process. In Proceedings of the 2021 5th National Conference on Advances in Enterprise Architecture (NCAEA), Mashhad, Iran, 1–2 December 2021; IEEE: New York, NY, USA, 2021; pp. 43–47. [CrossRef]
22. Zhu, J.; Viaud, G.; Hudelot, C. Improving Next-Application Prediction with Deep Personalized-Attention Neural Network. In Proceedings of the 2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA), Pasadena, CA, USA, 13–16 December 2021; IEEE: New York, NY, USA, 2021; pp. 1615–1622. [CrossRef]

23. Bellini, V.; Biancofiore, G.M.; Di Noia, T.; Sciascio, E.D.; Narducci, F.; Pomo, C. GUapp: A Conversational Agent for Job Recommendation for the Italian Public Administration. In Proceedings of the 2020 IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS), Bari, Italy, 27–29 May 2020; IEEE: New York, NY, USA, 2020; pp. 1–7. [CrossRef]
24. Yadalam, T.V.; Gowda, V.M.; Kumar, V.S.; Girish, D.; Namratha, M. Career Recommendation Systems using Content based Filtering. In Proceedings of the 2020 5th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 10–12 June 2020; IEEE: New York, NY, USA, 2020; pp. 660–665. [CrossRef]
25. Nigam, A.; Roy, A.; Singh, H.; Waila, H. Job Recommendation through Progression of Job Selection. In Proceedings of the 2019 IEEE 6th International Conference on Cloud Computing and Intelligence Systems (CCIS), Singapore, 19–21 December 2019; IEEE: New York, NY, USA, 2019; pp. 212–216. [CrossRef]
26. Jain, H.; Kakkar, M. Job Recommendation System based on Machine Learning and Data Mining Techniques using RESTful API and Android IDE. In Proceedings of the 2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, 10–11 January 2019; IEEE: New York, NY, USA, 2019; pp. 416–421. [CrossRef]
27. Zhou, Q.; Liao, F.; Ge, L.; Sun, J. Personalized Preference Collaborative Filtering: Job Recommendation for Graduates. In Proceedings of the 2019 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI), Leicester, UK, 19–23 August 2019; IEEE: New York, NY, USA, 2019; pp. 1055–1062. [CrossRef]
28. Mehta, M.; Derasari, R.; Patel, S.; Kakadiya, A.; Gandhi, R.; Chaudhary, S.; Goswami, R. A Service-Oriented Human Capital Management Recommendation Platform. In Proceedings of the 2019 IEEE International Systems Conference (SysCon), Orlando, FL, USA, 8–11 April 2019; IEEE: New York, NY, USA, 2019; pp. 1–8. [CrossRef]
29. Lin, Y.; Huang, Y.; Chen, P. Employment Recommendation Algorithm Based on Ensemble Learning. In Proceedings of the 2019 IEEE 1st International Conference on Civil Aviation Safety and Information Technology (ICCASIT), Kunming, China, 17–19 October 2019; IEEE: New York, NY, USA, 2019; pp. 267–271. [CrossRef]
30. Almalki, L. BERT-based Job Recommendation System Using LinkedIn Dataset. *J. Inf. Syst. Eng. Manag.* **2025**, *10*, 280–291. [CrossRef]
31. Hickey, P.J.; Erfani, A.; Cui, Q. Use of LinkedIn Data and Machine Learning to Analyze Gender Differences in Construction Career Paths. *J. Manag. Eng.* **2022**, *38*, 04022060. [CrossRef]
32. Panchasara, S.; Gupta, R.K.; Sharma, A. AI Based Job Recommedation System using BERT. In Proceedings of the 2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA), Pune, India, 18–19 August 2023; IEEE: New York, NY, USA, 2023; pp. 1–6. [CrossRef]
33. Li, M.; Chen, X.; Li, X.; Ma, B.; Vitanyi, P. The Similarity Metric. *IEEE Trans. Inf. Theory* **2004**, *50*, 3250–3264. [CrossRef]
34. Lahitani, A.R.; Permanasari, A.E.; Setiawan, N.A. Cosine similarity to determine similarity measure: Study case in online essay assessment. In Proceedings of the 2016 4th International Conference on Cyber and IT Service Management, Bandung, Indonesia, 26–27 April 2016; IEEE: New York, NY, USA, 2016; pp. 1–6. [CrossRef]
35. Rosnes, D.; Starke, A.D.; Trattner, C. Shaping the Future of Content-based News Recommenders: Insights from Evaluating Feature-Specific Similarity Metrics. In Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization, Cagliari, Italy, 1–4 July 2024; ACM: New York, NY, USA, 2024; pp. 201–211. [CrossRef]
36. Billsus, D. User Modeling for Adaptive News Access. *User Model.-User-Adapt. Interact.* **2000**, *10*, 147–180. [CrossRef]
37. Sanchan, N. Comparative Study on Automated Reference Summary Generation using BERT Models and ROUGE Score Assessment. *J. Curr. Sci. Technol.* **2024**, *14*, 26. . [CrossRef]
38. Shakil, H.; Farooq, A.; Kalita, J. Abstractive Text Summarization: State of the Art, Challenges, and Improvements. *Neurocomputing* **2024**, *603*, 128255. [CrossRef]
39. Garrido-Merchan, E.C.; Gozalo-Brizuela, R.; Gonzalez-Carvajal, S. Comparing BERT Against Traditional Machine Learning Models in Text Classification. *J. Comput. Cogn. Eng.* **2023**, *2*, 352–356. [CrossRef]
40. Wehnert, S.; Sudhi, V.; Dureja, S.; Kutty, L.; Shahania, S.; De Luca, E.W. Legal norm retrieval with variations of the bert model combined with TF-IDF vectorization. In Proceedings of the Eighteenth International Conference on Artificial Intelligence and Law, São Paulo, Brazil, 21–25 June 2021; ACM: New York, NY, USA, 2021; pp. 285–294. [CrossRef]


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.